Algoritmos de Ordenação (mergesort e quicksort)

Estruturas de Dados

adalberto@computacao.ufcg.edu.br

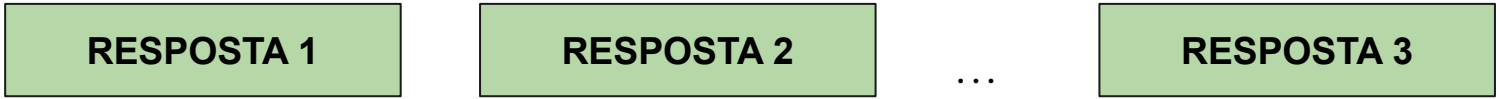
Divisão e conquista

PROBLEMA

Divisão e conquista

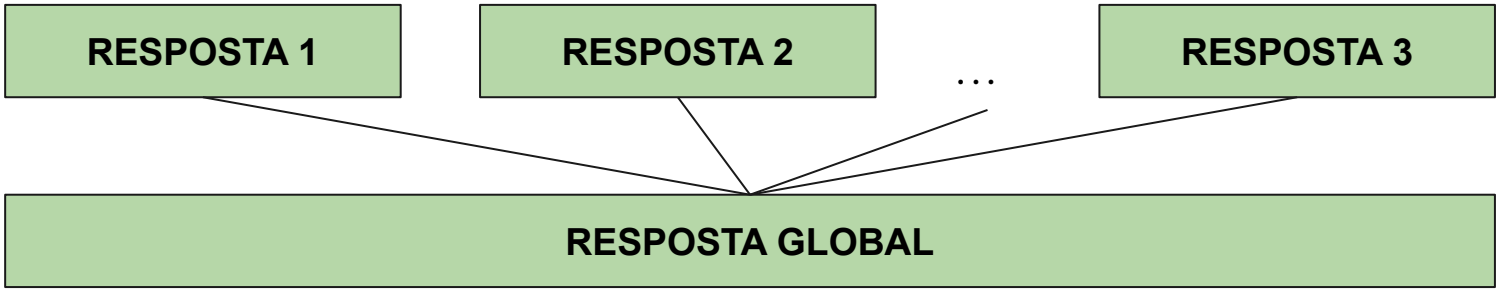
- Dividir
  - Problema grande → sub-problemas

Divisão e conquista



- Conquistar
  - Resolver sub-problemas (**recursão**)

Divisão e conquista



- Combinar
  - Combinar respostas dos sub-problemas

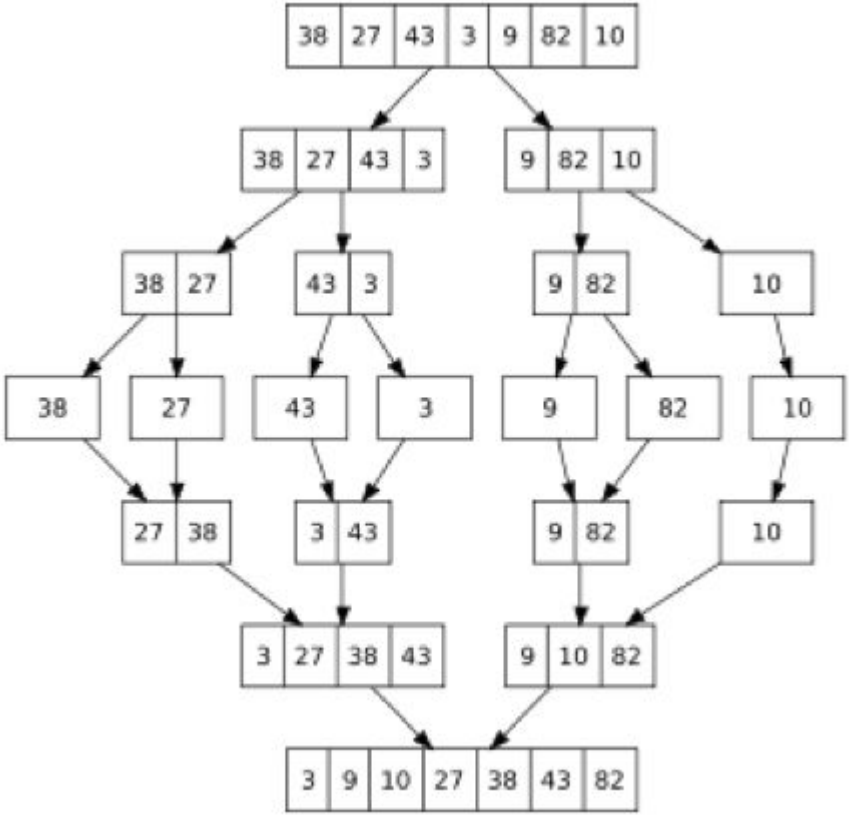
Divisão e conquista

- Mergesort

- Quicksort

Mergesort

Mergesort


38 27 43 3 9 82 10
38 27 43 3
9 82 10
38 27
43 3
9 82
10
38
27
43
3
9
82
10
27 38
3 43
9 82
10
3 27 38 43
9 10 82
3 9 10 27 38 43 82

Mergesort

```
merge_sort(A):
   if len(A) == 1:
      return
   else:
      merge_sort(A[1..n/2])
      merge_sort(A[n/2+1, n])
      merge_the_two_sorted_lists
```

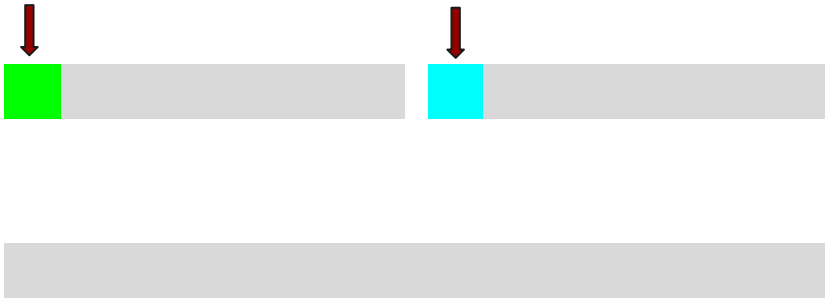
Mergesort

```
merge_sort(A):
    if len(A) == 1:
        return
    else:
        merge_sort(A[1..n/2])
        merge_sort(A[n/2+1, n])
        merge_the_two_sorted_lists
```

Implementação em Java com List!

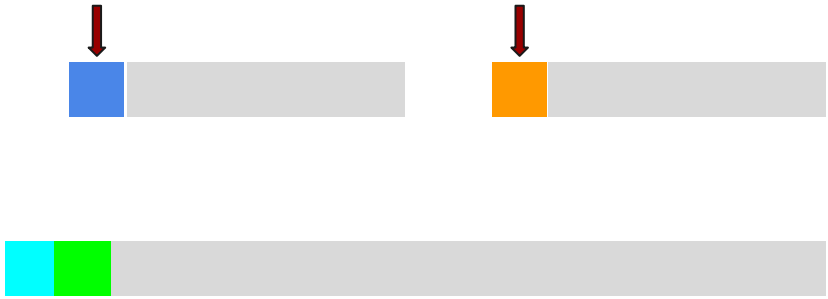
Merge de duas listas ordenadas

```
List merge(List left, List right) {
  List result = new List();
  while (left.length() > 0 and right.length() > 0) {
    if (left.first() ≤ right.first())
      result.add(left.removeFirst());
    else
      result.add(right.removeFirst());
  }
  if (left.length() > 0) result.append(left);
  if (right.length() > 0) result.append(right);
  return result;
}
```



Merge de duas listas ordenadas

```
List merge(List left, List right) {
  List result = new List();
  while (left.length() > 0 and right.length() > 0) {
    if (left.first() ≤ right.first())
      result.add(left.removeFirst());
    else
      result.add(right.removeFirst());
  }
  if (left.length() > 0) result.append(left);
  if (right.length() > 0) result.append(right);
  return result;
}
```



Merge de duas listas ordenadas (tempo?)

```
List merge(List left, List right) {
  List result = new List();
  while (left.length() > 0 and right.length() > 0) {
    if (left.first() ≤ right.first())
      result.add(left.removeFirst());
    else
      result.add(right.removeFirst());
  }
  if (left.length() > 0) result.append(left);
  if (right.length() > 0) result.append(right);
  return result;
}
```

Merge de duas listas ordenadas (tempo?)

```
List merge(List left, List right) {
  List result = new List();
  while (left.length() > 0 and right.length() > 0) {
    if (left.first() ≤ right.first())
      result.add(left.removeFirst());
    else
      result.add(right.removeFirst());
  }
  if (left.length() > 0) result.append(left);
  if (right.length() > 0) result.append(right);
  return result;
}
```

$\Rightarrow$ θ (len(left) + len(right))

Merge de duas listas ordenadas (tempo?)

```
List merge(List left, List right) {
  List result = new List();
  while (left.length() > 0 and right.length() > 0) {
    if (left.first() ≤ right.first())
      result.add(left.removeFirst());
    else
      result.add(right.removeFirst());
  }
  if (left.length() > 0) result.append(left);
  if (right.length() > 0) result.append(right);
  return result;
}
```

$\Rightarrow$ $\boldsymbol{\theta}$ (n)

Mergesort (simulação)

Mergesort (tempo)

```
merge_sort(A):
   if len(A) == 1:
      return
   else:
      merge_sort(A[1..n/2])
      merge_sort(A[n/2+1, n])
      merge_the_two_sorted_lists
```

Mergesort (tempo)

```
merge_sort(A):
    if len(A) == 1:
        return
    else:
        merge_sort(A[1..n/2])
        merge_sort(A[n/2+1, n])
        merge_the_two_sorted_lists
```

T(n)

T(n/2)

T(n/2)

Mergesort (tempo)

```
merge_sort(A):
    if len(A) == 1:
        return
    else:
        merge_sort(A[1..n/2])
        merge_sort(A[n/2+1, n])
        merge_the_two_sorted_lists
```

T(n)

c

T(n/2)

T(n/2)

θ(n)

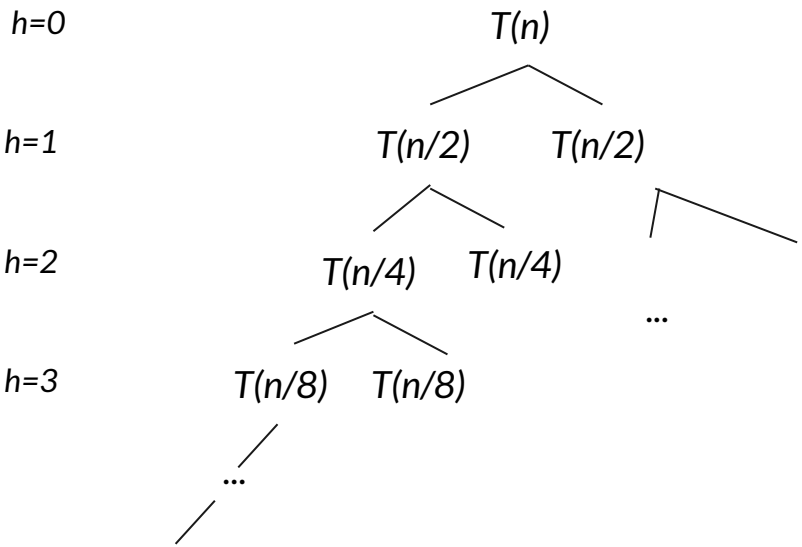
Mergesort (tempo)

```
merge_sort(A):
    if len(A) == 1:
        return
    else:
        merge_sort(A[1..n/2])
        merge_sort(A[n/2+1, n])
        merge_the_two_sorted_lists
```

$T(n) = 2T(n/2) + n$

Mergesort (cálculo do tempo)

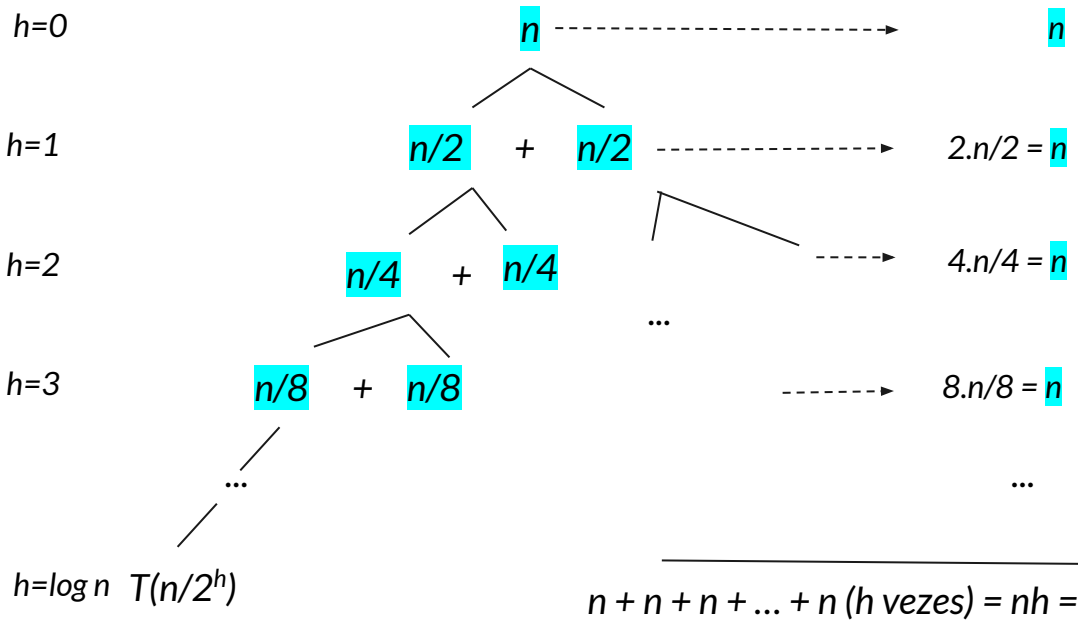
$T(n) = 2T(n/2) + n$



h $\quad T(n/2^h)=T(1) \Rightarrow n/2^h = 1 \Rightarrow 2^h = n \Rightarrow \log 2^h = \log n \Rightarrow h = \log n$

Mergesort (cálculo do tempo)

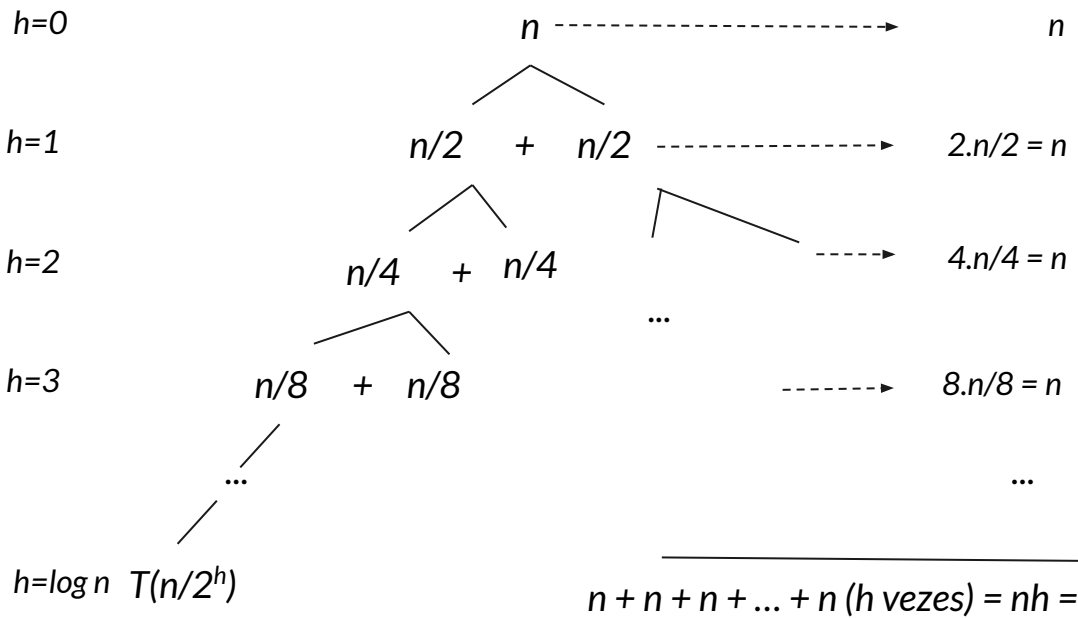
$T(n) = 2T(n/2) + n$



$n + n + n + \ldots + n$ (h vezes) $= nh = \theta(n \log n)$

Mergesort (cálculo do tempo)

$T(n) = 2T(n/2) + n$



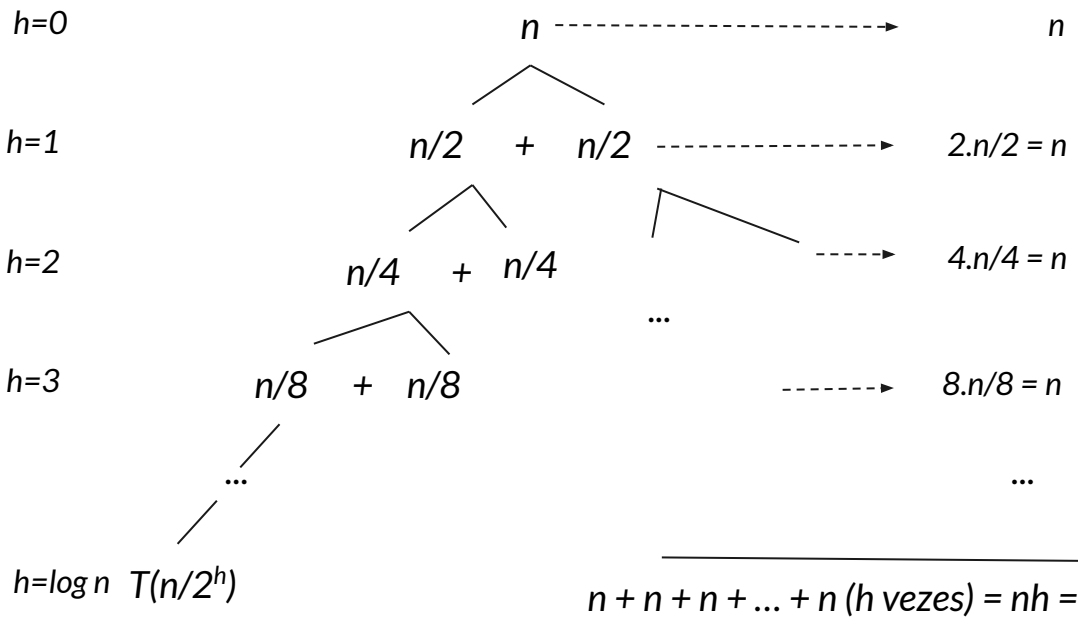
$a = 2$
$b = 2$
$f(n) = n$

$$n = n^{\log 2}$$

$$n + n + n + \ldots + n \text{ (h vezes)} = nh = \theta(n \log n)$$

Mergesort (cálculo do tempo)

$T(n) = 2T(n/2) + n$



$a = 2$
$b = 2$
$f(n) = n$

$$n = n^{\log 2}$$

$$\Downarrow$$

$$\theta(n \log n)$$

$$n + n + n + \ldots + n \text{ (h vezes)} = nh = \theta(n \log n)$$

Mergesort (propriedades)

Performance?

In-place?

Estável?

Mergesort (propriedades)

Performance?

In-place?

Estável?

```
List merge(List left, List right) {
  List result = new List();
  while (left.length() > 0 and right.length() > 0) {
    if (left.first() ≤ right.first())
      result.add(left.removeFirst());
    else
      result.add(right.removeFirst());
  }
  if (left.length() > 0) result.append(left);
  if (right.length() > 0) result.append(right);
  return result;
}
```

1 7 9 3 7 14

1 3 7 7 9 14

Mergesort (propriedades)

Performance?

In-place?

Estável?

```
List merge(List left, List right) {
  List result = new List();
  while (left.length() > 0 and right.length() > 0) {
    if (left.first() ≤ right.first())
      result.add(left.removeFirst());
    else
      result.add(right.removeFirst());
  }
  if (left.length() > 0) result.append(left);
  if (right.length() > 0) result.append(right);
  return result;
}
```

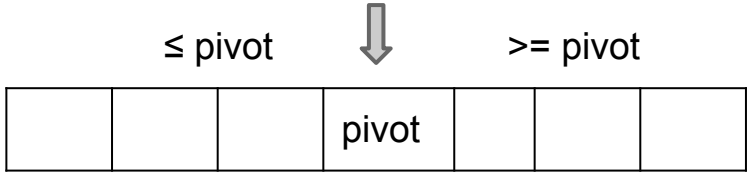
Quicksort

Marktoberdorf Summerschool 2006

Quicksort

1. Dividir

   Particionar vetor de acordo com a escolha de um pivot.



2. Conquistar

   Ordena recursivamente cada sub-vetor

3. Combinar

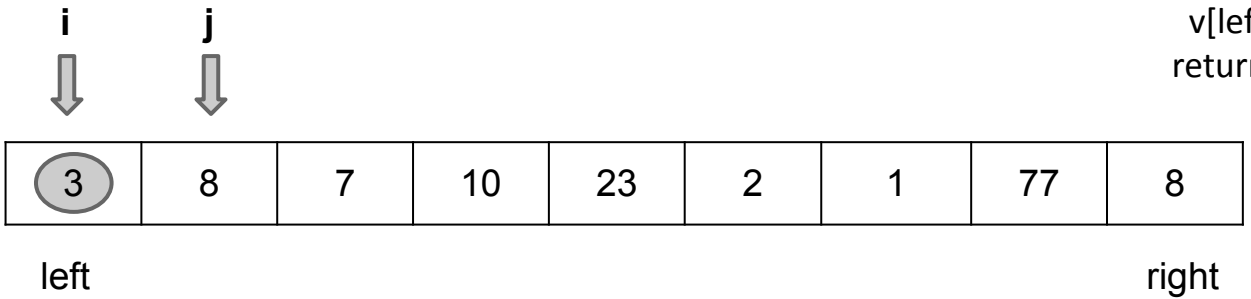
   Trivial

Quicksort

```
quickSort(int[] v, int left, int right)
    if (left < right)
        pivot = partition(v,left,right)
        quickSort(v,left,pivot-1)
        quickSort(v,pivot + 1, right)
```

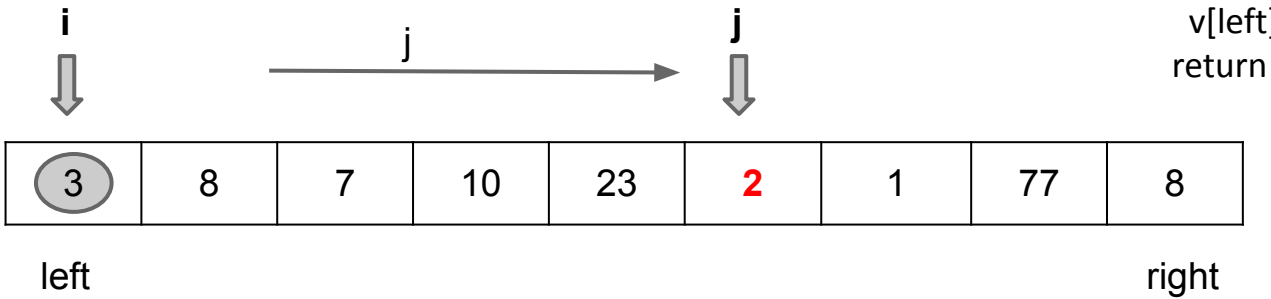
Partition

```
partition(v, left, right)
   pivot =  v[left]
   # position to place the pivot
   i = left
   for j = left+ 1 to right
     if (v[j] <= pivot)
        i += 1
        v[i],v[j] = v[j],v[i]

    v[left],v[i] = v[i],v[left]
   return i
```



Início do particionamento

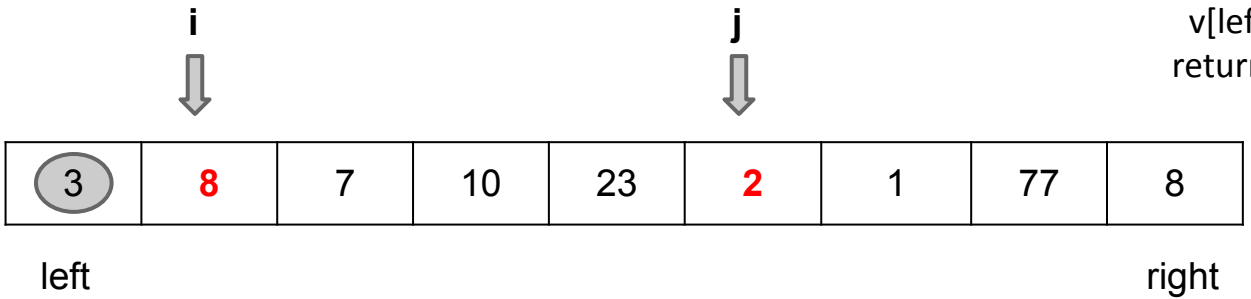
Partition




```
partition(v, left, right)
   pivot =  v[left]
   # position to place the pivot
   i = left
   for j = left+ 1 to right
     if (v[j] <= pivot)
        i += 1
        v[i],v[j] = v[j],v[i]

    v[left],v[i] = v[i],v[left]
   return i
```

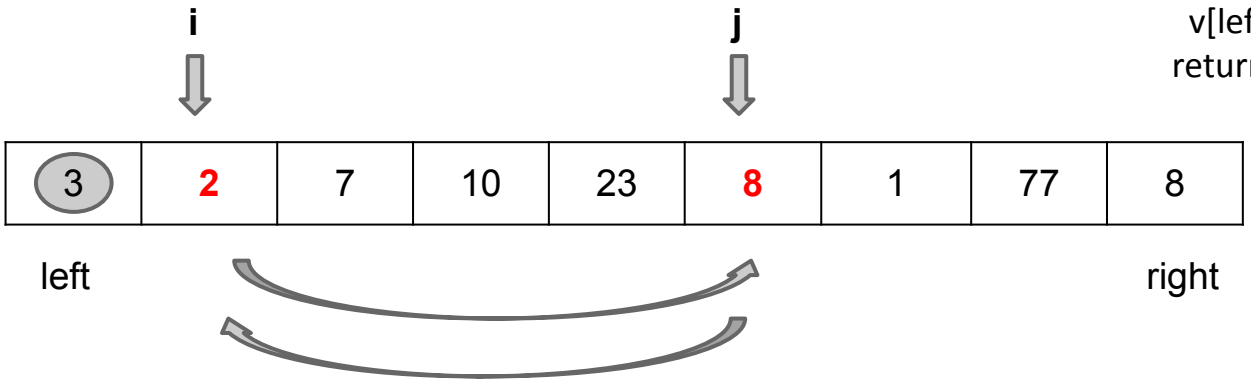
Partition

```
partition(v, left, right)
   pivot =  v[left]
   # position to place the pivot
   i = left
   for j = left+ 1 to right
     if (v[j] <= pivot)
        i += 1
        v[i],v[j] = v[j],v[i]

    v[left],v[i] = v[i],v[left]
   return i
```



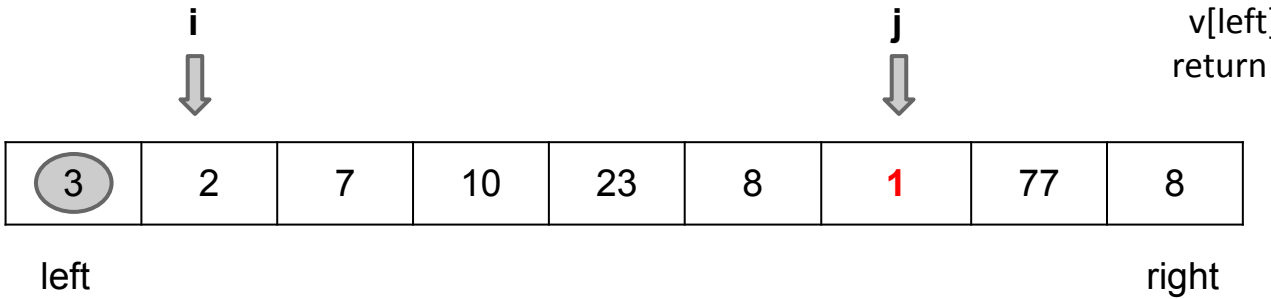
Partition

```
partition(v, left, right)
   pivot =  v[left]
   # position to place the pivot
   i = left
   for j = left+ 1 to right
     if (v[j] <= pivot)
       i += 1
       v[i],v[j] = v[j],v[i]

   v[left],v[i] = v[i],v[left]
  return i
```



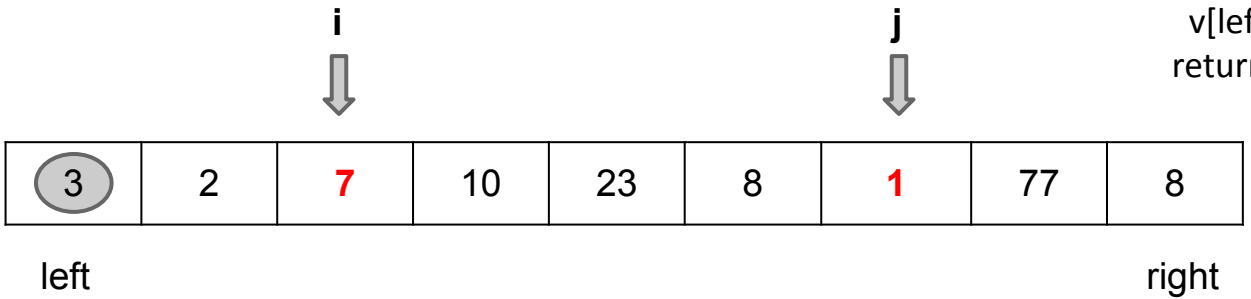
Partition

```
partition(v, left, right)
   pivot =  v[left]
   # position to place the pivot
   i = left
   for j = left+ 1 to right
     if (v[j] <= pivot)
        i += 1
        v[i],v[j] = v[j],v[i]

    v[left],v[i] = v[i],v[left]
   return i
```



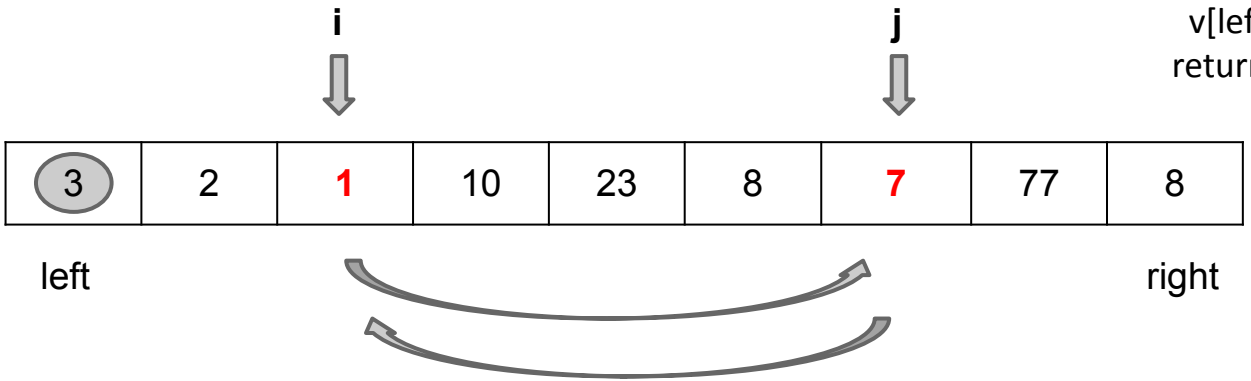
Partition

```
partition(v, left, right)
   pivot =  v[left]
   # position to place the pivot
   i = left
   for j = left+ 1 to right
     if (v[j] <= pivot)
       i += 1
        v[i],v[j] = v[j],v[i]

    v[left],v[i] = v[i],v[left]
   return i
```



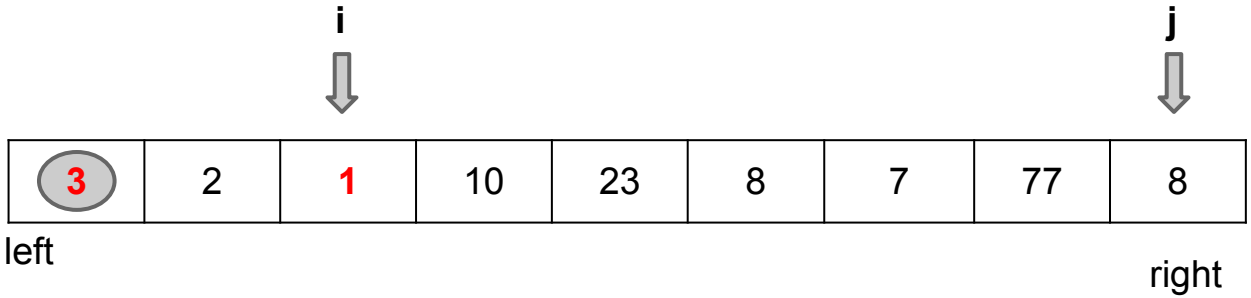
Partition

```
partition(v, left, right)
   pivot =  v[left]
   # position to place the pivot
   i = left
   for j = left+ 1 to right
     if (v[j] <= pivot)
       i += 1
       v[i],v[j] = v[j],v[i]

   v[left],v[i] = v[i],v[left]
  return i
```



Partition

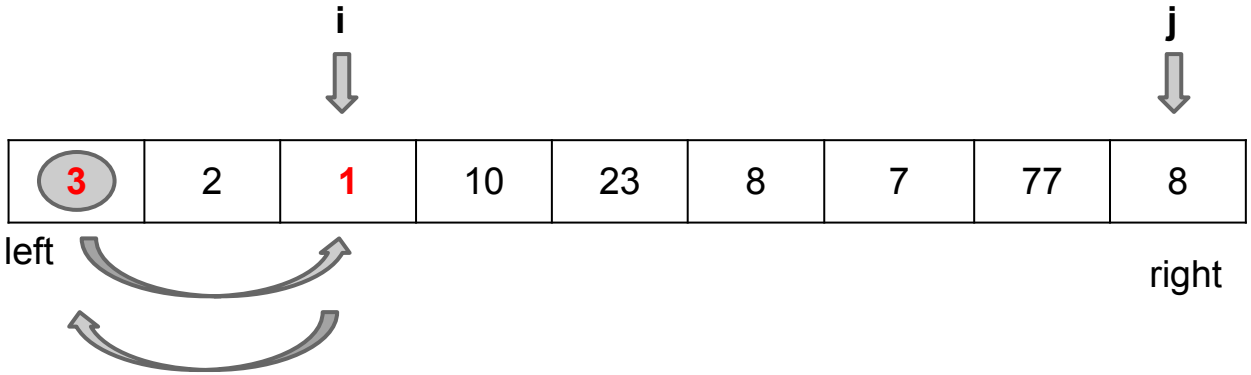
```
partition(v, left, right)
   pivot =  v[left]
   # position to place the pivot
   i = left
   for j = left+ 1 to right
     if (v[j] <= pivot)
        i += 1
        v[i],v[j] = v[j],v[i]

   v[left],v[i] = v[i],v[left]
  return i
```



Fim da iteração (j == right)

Partition

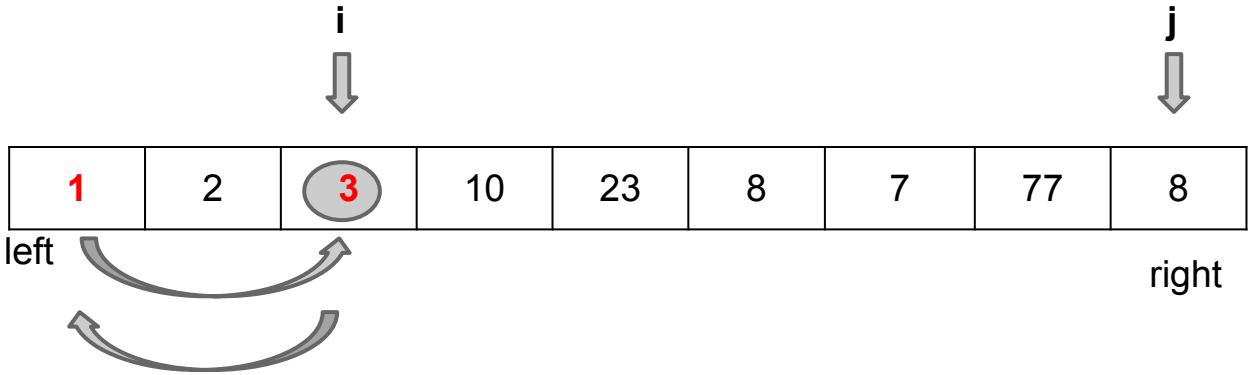
```
partition(v, left, right)
   pivot =  v[left]
   # position to place the pivot
   i = left
   for j = left+ 1 to right
     if (v[j] <= pivot)
       i += 1
       v[i],v[j] = v[j],v[i]

   v[left],v[i] = v[i],v[left]
  return i
```



Trocar left por i.

Partition

```
partition(v, left, right)
   pivot =  v[left]
   # position to place the pivot
   i = left
   for j = left+ 1 to right
     if (v[j] <= pivot)
       i += 1
       v[i],v[j] = v[j],v[i]

   v[left],v[i] = v[i],v[left]
  return i
```



Trocar left por i.

Partition

1	2	3	10	23	8	7	77	8

left

right

Fim do particionamento

- Respeita a invariante?

Partition

Próximo particionamento

1	2	3	10	23	8	7	77	8
left	right		left					right

quickSort(v,left,pivot-1)

quickSort(v,pivot + 1, right)

...

Partition (tempo)

```
partition(v, left, right)
    pivot =  v[left]
    # position to place the pivot
    i = left
    for j = left+ 1 to right
      if (v[j] <= pivot)
         i += 1
         v[i],v[j] = v[j],v[i]

     v[left],v[i] = v[i],v[left]
   return i
```

Partition (tempo)

```
partition(v, left, right)
    pivot =  v[left]
    # position to place the pivot
    i = left
    for j = left+ 1 to right
      if (v[j] <= pivot)
         i += 1
         v[i],v[j] = v[j],v[i]

     v[left],v[i] = v[i],v[left]
   return i
```

$\Rightarrow$ $\boldsymbol{\theta}$ (n)

Quicksort (simulação)

Quicksort (tempo)

```
quickSort(int[] v, int left, int right)
    if (left < right)
        pivot = partition(v,left,right)
        quickSort(v,left,pivot-1)
        quickSort(v,pivot + 1, right)
```

$$T(n) = T(|left|) + T(|right|) + \Theta(n)$$

Quicksort (tempo)

```
quickSort(int[] v, int left, int right)
    if (left < right)
        pivot = partition(v,left,right)
        quickSort(v,left,pivot-1)
        quickSort(v,pivot + 1, right)
```

$$T(n) = T(|left|) + T(|right|) + \Theta(n)$$

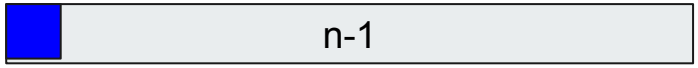
Depende do PIVOT!

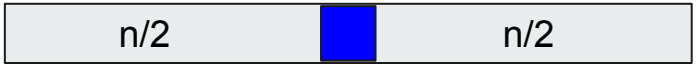
Quicksort (tempo)

```
quickSort(int[] v, int left, int right)
    if (left < right)
        pivot = partition(v,left,right)
        quickSort(v,left,pivot-1)
        quickSort(v,pivot + 1, right)
```

$$T(n) = T(|left|) + T(|right|) + \Theta(n)$$

Depende do PIVOT!





Quicksort (pior caso)

```
quickSort(int[] v, int left, int right)
    if (left < right)
        pivot = partition(v,left,right)
        quickSort(v,left,pivot-1)
        quickSort(v,pivot + 1, right)
```

$$T(n) = \Theta(1) + T(n-1) + \Theta(n)$$

$$\Downarrow$$

$$\Theta(n^2)$$

Quicksort (melhor caso)

```
quickSort(int[] v, int left, int right)
    if (left < right)
        pivot = partition(v,left,right)
        quickSort(v,left,pivot-1)
        quickSort(v,pivot + 1, right)
```

$$T(n) = 2T(n/2) + \Theta(n)$$

$$\Downarrow$$

$$\Theta(n \log n)$$

Quicksort

Como garantir um particionamento razoável?

- Embaralhar a entrada
- Escolher um pivot aleatoriamente

Quicksort (propriedades)

Performance?

In-place?

Estável?

Quicksort (propriedades)

Performance?

In-place?

Estável?

```
partition(v, left, right)
    pivot =  v[left]
    # position to place the pivot
    i = left
    for j = left+ 1 to right
      if (v[j] <= pivot)
         i += 1
         v[i],v[j] = v[j],v[i]

     v[left],v[i] = v[i],v[left]
   return i
```

Comparação

Algoritmo	melhor tempo	tempo médio	pior tempo	espaço
Bubble sort	O(n)	O(nˆ2)	O(nˆ2)	in-place
Selection sort	O(nˆ2)	O(nˆ2)	O(nˆ2)	in-place
Insertion sort	O(n)	O(nˆ2)	O(nˆ2)	in-place
Merge sort	O(n log n)	O(n log n)	O(n log n)	O(n)
Quick sort	O (n log n)	O (n log n)	O(n^2)	Tamanho da pilha de recursão. O(log n)

Quicksort (particionamento em 10% e 90%)

$$T(n) = T(n/10) + T(9n/10) + n$$

$$\Downarrow$$

$$\boldsymbol{\theta}\,(n \log_{10/9} n)$$